AI PBL · FINAL REVIEW

Intelligent Traffic Signal Control Using Fuzzy Logic

From pitch to finished prototype: a fuzzy controller tuned by optimization, tested on traffic it never trained on.

TOPIC #15 | FOUNDATIONS OF AI & OPTIMIZATION | 24TU03MJM3 | FINAL REVIEW

D. Ritheek
25WU0102057

M. Srilakshmi Reddy
25WU0102140

B. Deekshan Raj
25WU0102037

P. Priyakaran Reddy
25WU0101090

G. Madhan
25WU0102344

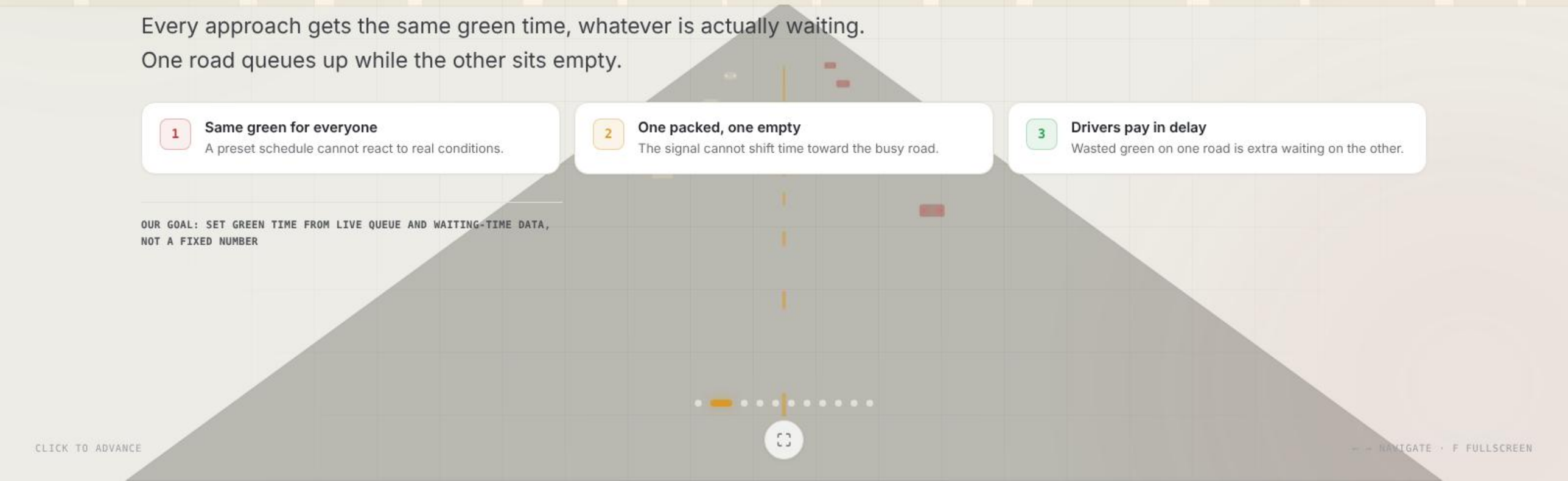
PROBLEM STATEMENT

Fixed timers can't see the traffic

Every approach gets the same green time, whatever is actually waiting.
One road queues up while the other sits empty.

1. **Same green for everyone**
   A preset schedule cannot react to real conditions.
2. **One packed, one empty**
   The signal cannot shift time toward the busy road.
3. **Drivers pay in delay**
   Wasted green on one road is extra waiting on the other.

OUR GOAL: SET GREEN TIME FROM LIVE QUEUE AND WAITING-TIME DATA, NOT A FIXED NUMBER



— PROGRESS

Since Review 1

Review 1
18 SEP
Topic locked, pitch, fuzzy controller stub

Core build
Mamdani controller, queue simulator, GA tuning

3D replay
Interactive side-by-side visualiser

v2 upgrade
3 inputs, 27 rules, GA / PSO / SA, 120 scenarios

Final review
FINAL
Finished prototype and measured results

- ✓ **Fuzzy controller** — Built and verified.
- ✓ **Optimizer tuning** — GA, PSO and SA compared.
- ✓ **Benchmark** — Against 3 baselines.
- ✓ **Prototype** — CLI, web app, 3D replay.

WORKING

How the controller decides



Crisp inputs
Own q, wait, opp. q

Fuzzification
Maps to fuzzy sets

Rule base
27 IF-THEN rules

Defuzzification
Centroid method

Crisp output
Green time, 10-60 s

IF

Example rule

IF own queue is High AND opposing queue is Low THEN green time is **Long**

The prototype prints which rules fired for every decision, so each green time can be explained.

COURSE LINK: RULE BASE = KNOWLEDGE REPRESENTATION (WK 1) · MAMDANI INFERENCE AND DEFUZZIFICATION = FUZZY LOGIC (WK 9)



WHAT CHANGED

From v1 to v2

ASPECT	V1	V2
Controller inputs	Own queue, own wait	**+ opposing queue (3 inputs)**
Rule base	9 rules	**27 rules**
Tuned parameters	3 membership centres	**All membership breakpoints**
Demand model	Constant arrival rates	**Plus time-varying profiles**
Training scenarios	3 hand-picked	**40 generated, 20 for validation**
Baselines	One fixed 30 s timer	**Plus best-fixed and actuated rule**

V1 WAS BLIND TO THE OTHER ROAD. V2 SEES BOTH, AND IS TESTED ON DATA IT NEVER TRAINED ON

— DATA AND TRAINING

Generated traffic, strictly split

40
TRAIN · TUNE THE CONTROLLER

20
VALIDATION · PICK THE FINAL ONE

60
TEST · SCORED EXACTLY ONCE

6 **Demand profiles**
Includes time-varying demand: rush-hour ramps, sudden surges and lulls.

3 **Optimizers, equal budget**
GA, PSO and SA: 2000 evaluations each, 27 runs in total.

ALL DATA IS SIMULATED TRAFFIC — THERE IS NO REAL-WORLD DATASET · SEEDS ARE FIXED, SO EVERY RUN IS REPRODUCIBLE

HOW WE TRAINED IT

Fuzzy decides, GA tunes

1 **Build candidate**
A list of numbers (all membership breakpoints) becomes a fuzzy controller.

2 **Test in simulator**
Run it on the 40 training scenarios. Average delay is its score, lower is better.

3 **Evolve**
The GA keeps the best, crosses and mutates them. Budget: 2000 evaluations, same for PSO and SA.

4 **Select and test**
Pick the winner on 20 validation scenarios, score once on 60 unseen test scenarios.

GA WRAPS AROUND THE FUZZY CONTROLLER

PROTOTYPE

A working prototype, not just a report

CLI **Explainable command line**
predict.py returns the recommended green time and the rules that fired.

WEB **Local web app**
Pick a scenario and controller. The real Python simulator runs behind it, offline.

3D **3D replay**
Fixed timer vs our controller, switchable at the same simulated second.

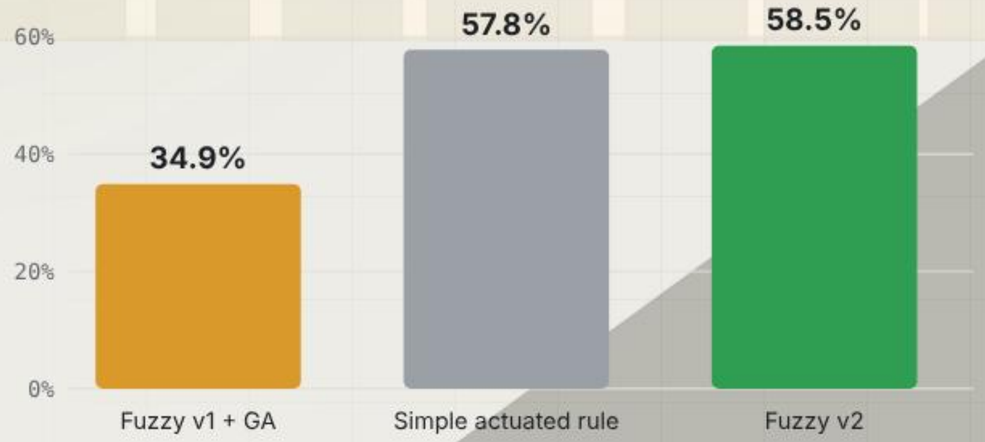
23 **Tested and reproducible**
23 unit tests pass. One command reproduces every result from fixed seeds.

LIVE DEMO: HTTP://LOCALHOST:8000 · RUNS FULLY OFFLINE

RESULTS

60 unseen scenarios, 58.5% less delay

MEAN DELAY REDUCTION VS FIXED 30 S TIMER



+42.4%

v2 vs v1

95% CI 36.2 to 48.5

59 wins · 0 losses · 1 tie

+4.3%

v2 vs simple actuated rule

95% CI 2.4 to 6.5

31 wins · 14 losses · 15 ties

TEST SET: 60 GENERATED SCENARIOS × 3 FRESH SEEDS, EVALUATED ONCE · BARS SHOW MEAN PER-SCENARIO IMPROVEMENT

HONEST FINDINGS

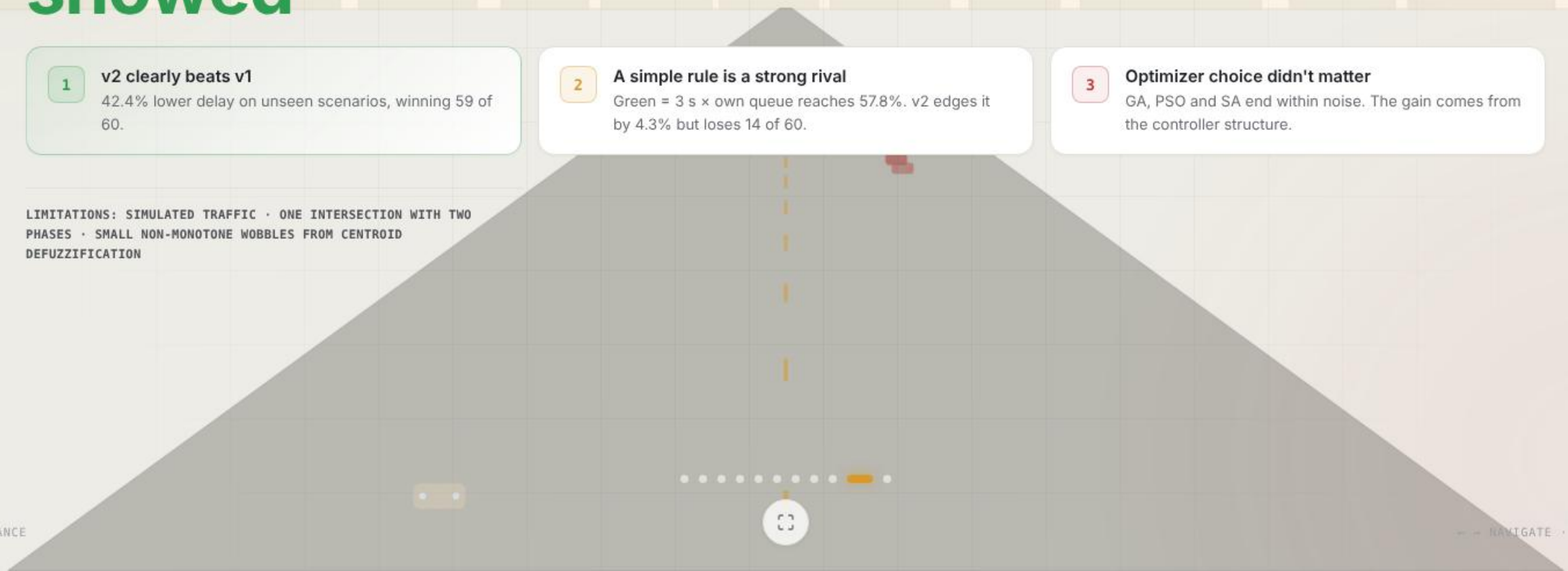
What the tests actually showed

1 **v2 clearly beats v1**
42.4% lower delay on unseen scenarios, winning 59 of 60.

2 **A simple rule is a strong rival**
Green = 3 s × own queue reaches 57.8%. v2 edges it by 4.3% but loses 14 of 60.

3 **Optimizer choice didn't matter**
GA, PSO and SA end within noise. The gain comes from the controller structure.

LIMITATIONS: SIMULATED TRAFFIC · ONE INTERSECTION WITH TWO PHASES · SMALL NON-MONOTONE WOBBLES FROM CENTROID DEFUZZIFICATION



— CONCLUSION

Adaptive control works

58.5%
V2 VS FIXED TIMER

42.4%
V2 VS V1

4.3%
V2 VS SIMPLE RULE

Our fuzzy controller matches adaptive control with explainable rules, and v2 improves v1 by 42%. The simple rule is a strong baseline, and our limits are simulated data and one two-phase intersection. Thank you — happy to take questions.

TOPIC #15 · 24TU03MJM3 · FINAL REVIEW

